

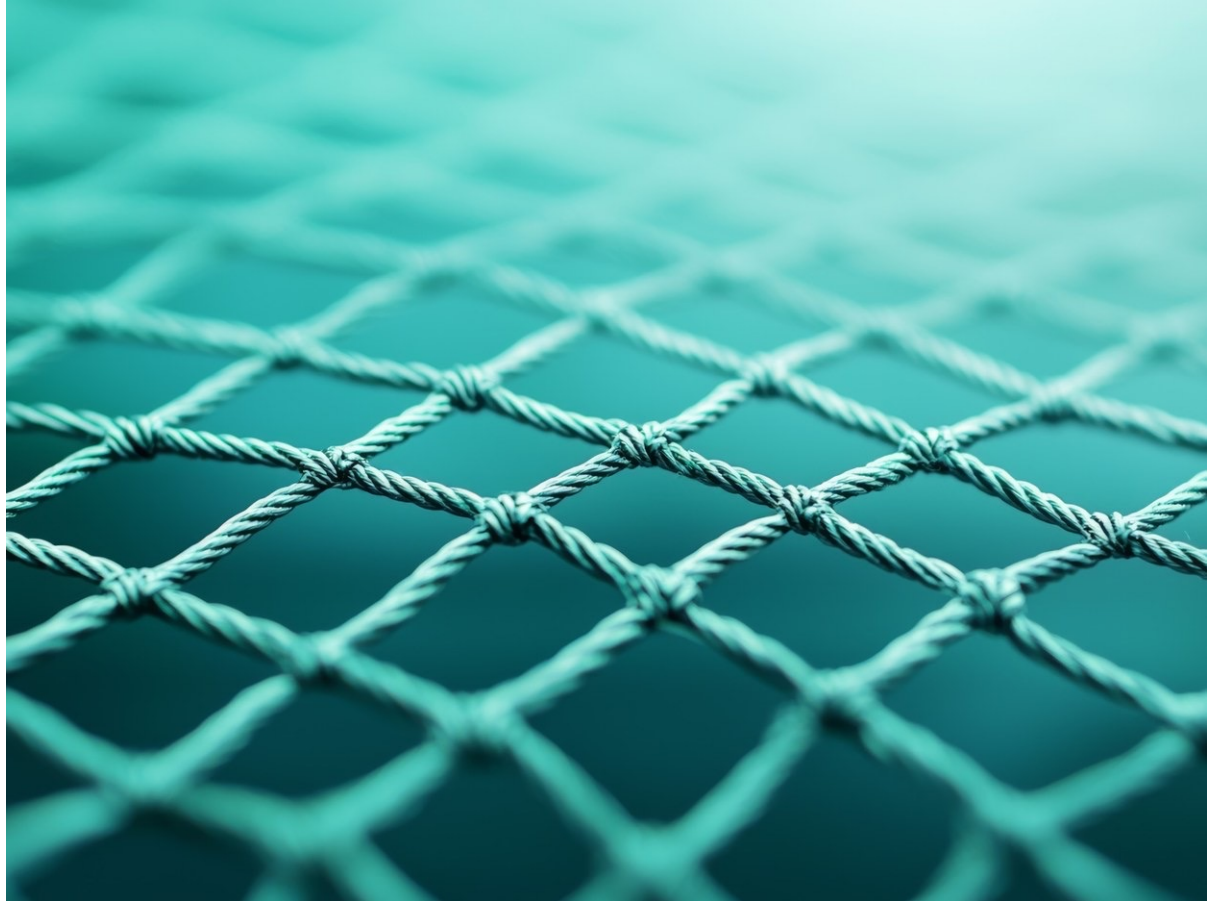
Monitoring





Observability in Modern Systems

- Observability is being able to tell what is happening inside a system just from the data it emits. In a distributed system that matters because you cannot attach a debugger to twenty services during an incident.
 - Observability is about system introspection
 - Explains behavior without direct access
 - Critical for distributed systems
 - Relies on telemetry from the system



Monitoring vs Observability

- Monitoring answers questions you already knew to ask; observability helps with the failures you did not predict. A dashboard catches the disk filling up, observability is how you explain a slowdown nobody saw coming.
 - Monitoring is reactive
 - Observability is exploratory
 - Monitoring answers known questions
 - Observability enables unknown questions

Monitoring vs observability

Monitoring

- known questions
- thresholds and checks
- reactive

Observability

- unknown failures
- explore the data
- proactive

Why Logs and Metrics Are Not Enough

- Logs and metrics tell you what happened and how often, but not how one request moved through the system. When a checkout is slow, neither can point to which of eight services actually caused it.
 - Metrics show trends
 - Logs show events
 - Neither shows request paths
 - Neither explains dependencies

Why logs and metrics fall short

Logs + metrics show

- trends over time
- individual events

They cannot show

- one request's path
- how services depend

The Distributed Systems Problem

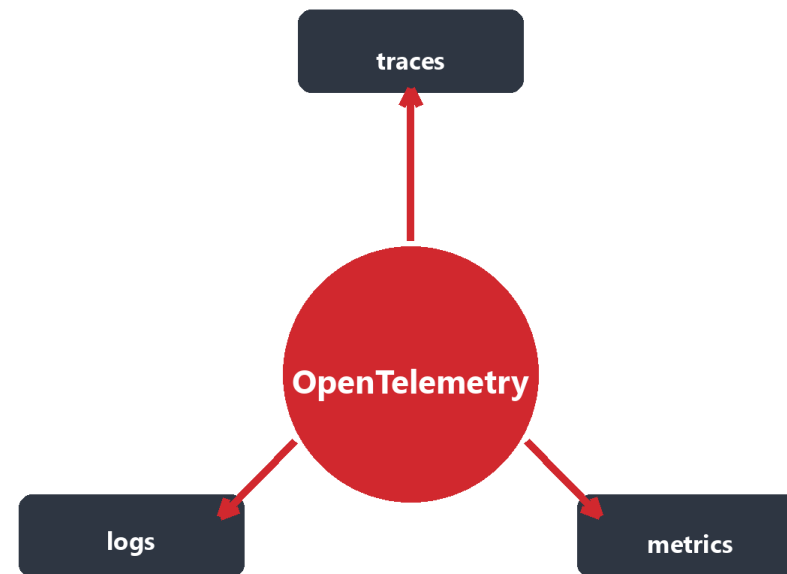
- Modern apps are many small services talking over the network, so failures emerge from the interactions, not one box. Latency compounds across hops, and the real root cause hides between services.
 - Requests span services
 - Failures propagate
 - Latency compounds
 - Root cause becomes unclear



What Is OpenTelemetry

- OpenTelemetry is the open, vendor-neutral standard for generating telemetry from your code. One set of instrumentation produces traces, metrics, and logs that can flow to any backend you choose.
 - Vendor-neutral standard
 - Open-source CNCF project
 - Cross-language support
 - Exports to any backend

■ What is OpenTelemetry



One standard, any backend

The Three Signals

- OpenTelemetry covers three signals, and each answers a different question. Traces show a request's path, metrics show the trends, and logs show the events, and together they explain behavior.
 - Traces show request flows
 - Metrics show numerical behavior
 - Logs show events
 - Signals complement each other

The three signals

Traces

- request flow

Metrics

- numeric trends

Logs

- events

What Is a Trace

- A trace is the full story of one request as it travels through the system. It breaks the work into spans that form a tree, so you can see where latency builds up end to end.
 - One trace per request
 - Spans form a tree
 - Parent-child relationships
 - Latency visible end-to-end

■ A trace is a tree of spans

GET /checkout

auth

inventory

payment

one trace per request



What Is a Span

- A span is one unit of work inside a trace, with a start time, an end time, and metadata. The line that says inventory lookup took 380ms and errored is a span.
 - Start and end time
 - Belongs to one trace
 - Can have children
 - Records errors

What a span is

✓ Start and end time

✓ Belongs to one trace

✓ Can have children

✓ Records errors

Why Tracing Exists

- Tracing exists to explain what metrics and logs cannot: causality across service boundaries. When a request is slow, the trace shows exactly where the time went and which call failed.
 - Explains slow requests
 - Explains failures
 - Explains dependencies
 - Enables root cause analysis

Why tracing exists

✓ Explains slow requests

✓ Explains failures

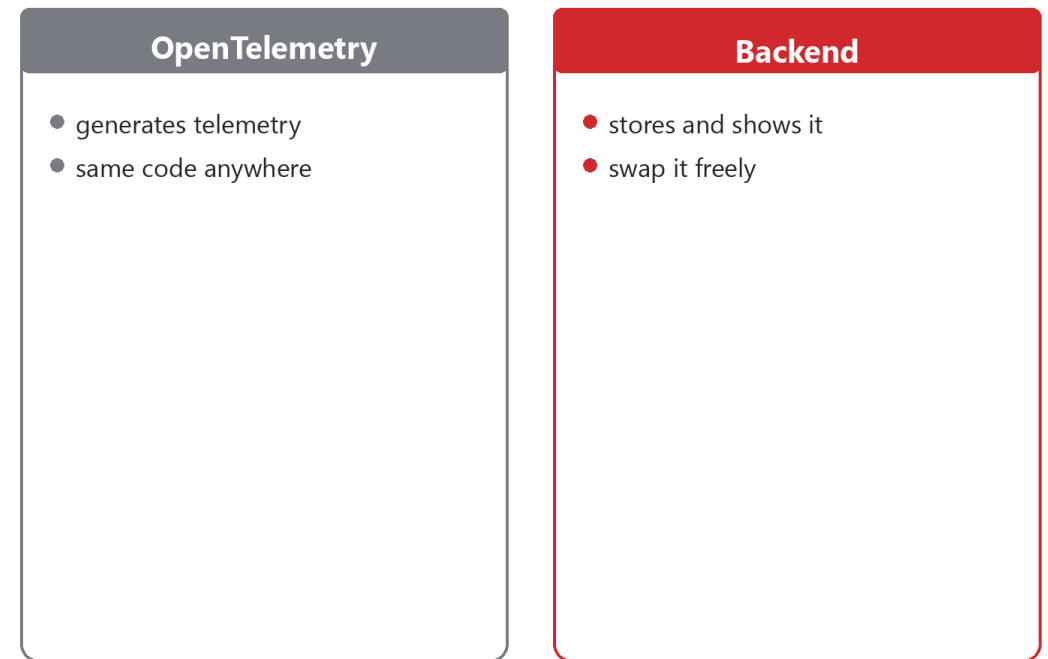
✓ Explains dependencies

✓ Enables root cause

OTEL Is Vendor Neutral

- OpenTelemetry only generates telemetry; it does not store or visualize it. That separation means the same instrumentation in your code works no matter which backend you point it at.
 - No vendor lock-in
 - Works with many APM platforms
 - Same code across environments
 - Backends can change freely

■ OTEL is vendor neutral



Why Standards Matter

- Before a standard, every vendor needed its own agent and instrumentation. A shared standard lets tools interoperate, so switching backends does not mean re-instrumenting your whole fleet.
 - Avoids proprietary agents
 - Enables tool portability
 - Reduces engineering overhead
 - Improves ecosystem growth

Why standards matter

- ✓ No proprietary agents
- ✓ Tool portability
- ✓ Less engineering overhead
- ✓ Healthier ecosystem

OTEL in Cloud Native Systems

- Cloud-native systems scale and move constantly, and instances are short-lived. Observability has to attach itself automatically, because no one can hand-configure tracing for pods that live minutes.
 - Services scale horizontally
 - Instances are short-lived
 - Infrastructure is abstracted
 - Tracing must be automatic

■ OTEL in cloud native

- ✓ Services scale out
- ✓ Instances are short-lived
- ✓ Infrastructure abstracted
- ✓ Tracing must be automatic

Telemetry as a Control Plane

- Telemetry is the feedback loop between your systems and the people running them. It drives incident response, capacity planning, and automation, not just the graphs on a wall.
 - Supports incident response
 - Enables capacity planning
 - Drives automation
 - Feeds reliability engineering

■ Telemetry as a control plane

Incident response

Capacity planning

Automation

Reliability

OTEL vs APM Agents

- Traditional APM agents lock you to one vendor's platform. OTEL decouples the instrumentation from the analysis, so you can change the backend without touching your code.
 - APM agents are proprietary
 - OTEL is open standard
 - OTEL supports multiple backends
 - Instrumentation is portable

OTEL vs APM agents

APM agents	OpenTelemetry
• vendor proprietary • tightly coupled	• open standard • portable, many backends

What OTEL Does Not Do

- It helps to be clear about the boundary: OTEL does not store data, draw dashboards, or fire alerts. It produces and exports telemetry, and leaves storage and visualization to tools like Tempo and Grafana.
 - No built-in dashboards
 - No built-in alerting
 - No data storage
 - Focuses only on collection

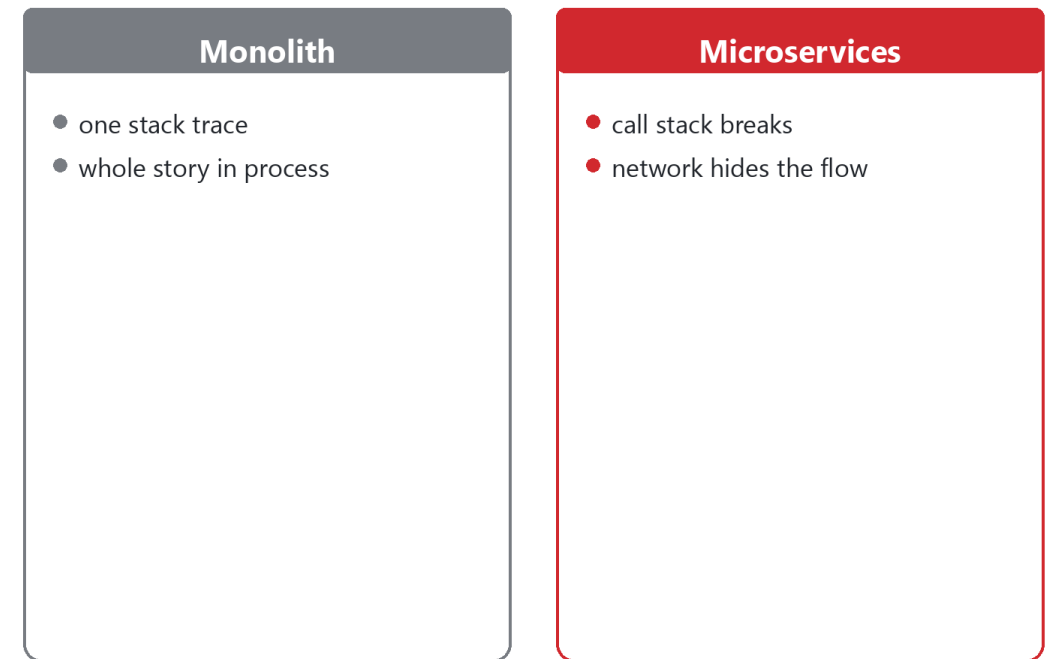
What OTEL does NOT do

- ✓ No built-in dashboards
- ✓ No built-in alerting
- ✓ No data storage
- ✓ Collection only

Why Tracing Became Critical

- In a monolith a stack trace told the whole story; microservices shattered that. Once a request crosses the network the call stack disappears, and tracing is what reconstructs the path.
 - Monoliths had single stack traces
 - Microservices break call stacks
 - Network hides execution flow
 - Tracing reconstructs execution paths

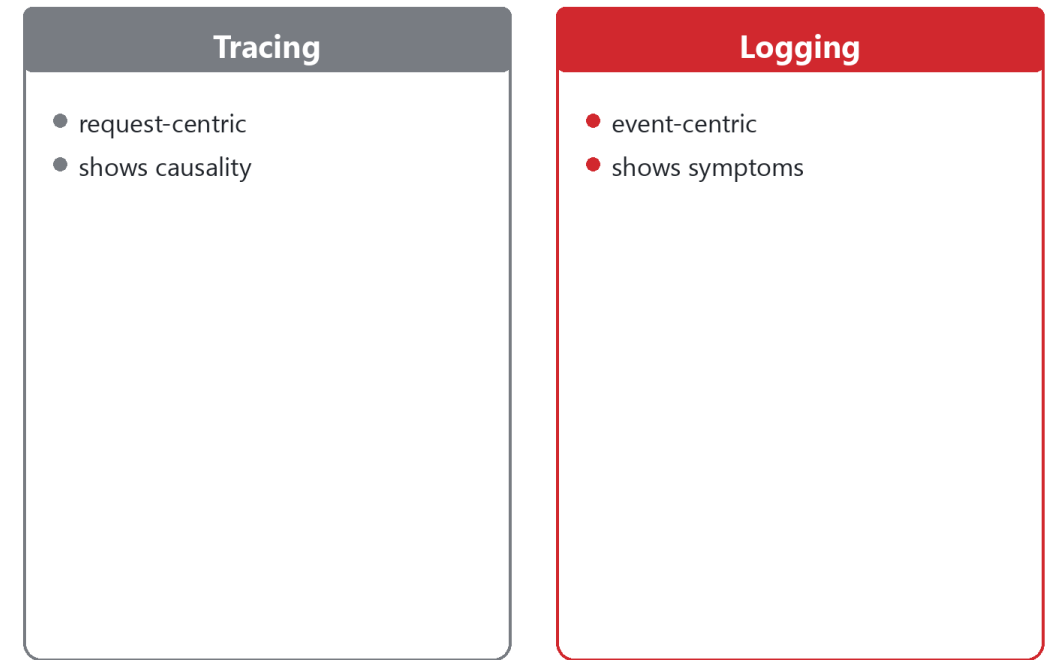
Why tracing became critical



Tracing vs Logging

- Tracing gives you a structured, request-centric view; logging gives you unstructured events. A trace shows causality across services, while a log line shows a single symptom in one of them.
 - Tracing is request-centric
 - Logging is event-centric
 - Tracing shows causality
 - Logging shows symptoms

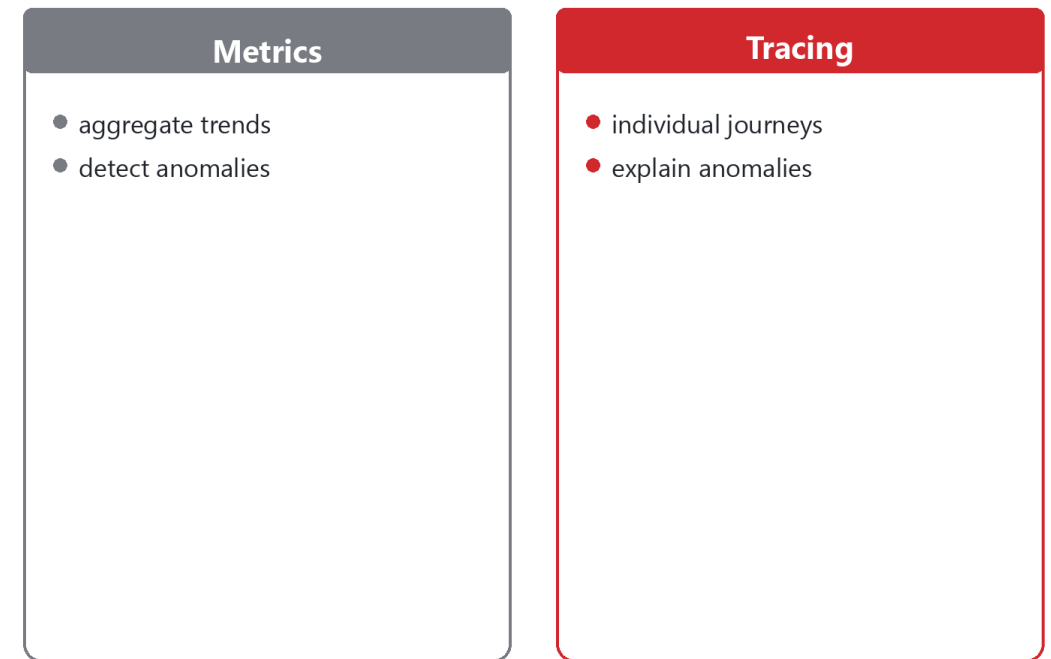
Tracing vs logging



Tracing vs Metrics

- Metrics aggregate everyone's experience into a trend; a trace shows one user's actual journey. Metrics tell you the p99 jumped, the trace tells you why for a specific slow request.
 - Metrics show trends
 - Tracing shows individual journeys
 - Metrics detect anomalies
 - Tracing explains anomalies

Tracing vs metrics



OTEL Core Components

- OTEL is a small pipeline of parts that hand telemetry along. Instrumentation creates it, the SDK processes it, the collector moves it, and exporters deliver it to a backend.
 - Instrumentation libraries
 - SDKs
 - Collectors
 - Exporters
 - Backends

■ OTEL core components



Instrumentation Libraries

- Instrumentation libraries hook into the frameworks you already use and emit spans for you. Because they are auto-injected, you get traces for HTTP and database calls with almost no code changes.
 - Framework specific
 - Language specific
 - Often auto-injected
 - Minimal code changes

Instrumentation libraries

 Framework specific

 Language specific

 Auto-injected

 Minimal code changes

OTEL SDK

- The SDK lives inside your application process and does the real work of telemetry. It creates tracers, manages each span's lifecycle, applies sampling, and hands data to the exporters.
 - Creates tracers and meters
 - Manages span lifecycle
 - Handles sampling
 - Sends data to exporters

The OTEL SDK

✓ Creates tracers and meters

✓ Manages span lifecycle

✓ Handles sampling

✓ Sends to exporters

OTEL Collector

- The Collector is a separate service that receives, processes, and forwards telemetry. Putting it out of process gives you one place to transform and route data before it reaches a backend.
 - Runs out of process
 - Receives OTLP
 - Can transform data
 - Forwards to backends

■ The OTEL Collector



runs out of process

Why Use a Collector

- A collector decouples your apps from your vendors. Apps just send to the collector, and it handles sampling, enrichment, and routing, so swapping backends never touches application code.
 - Decouples apps from vendors
 - Enables sampling control
 - Supports multiple backends
 - Simplifies network topology

Why use a collector

- ✓ Decouples apps from vendors
- ✓ Central sampling control
- ✓ Multiple backends
- ✓ Simpler topology

OTEL Exporters

- Exporters decide how telemetry leaves the system, translating it into a backend's protocol. OTLP is the native one, with Jaeger, Zipkin, and vendor-specific exporters available too.
 - OTLP exporter
 - Jaeger exporter
 - Zipkin exporter
 - Vendor specific exporters

OTEL exporters

OTLP

Jaeger

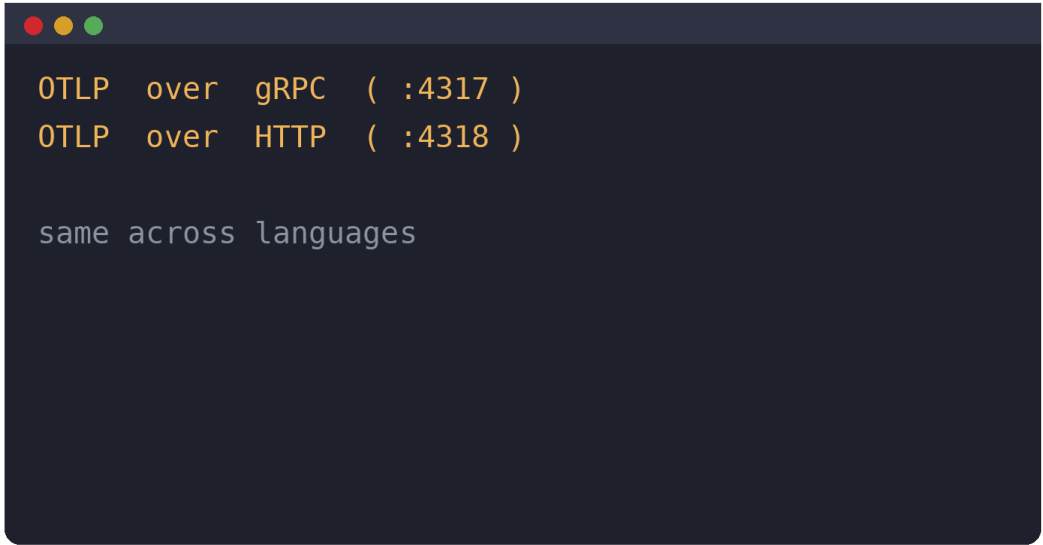
Zipkin

Vendor-specific

OTLP Protocol

- OTLP is OpenTelemetry's own wire protocol for shipping telemetry. It runs over gRPC or HTTP, is the same across languages, and is the default you use unless you have a reason not to.
 - HTTP or gRPC
 - Binary encoding
 - Standard across languages
 - Default OTEL protocol

OTLP protocol



```
OTLP over gRPC ( :4317 )
OTLP over HTTP ( :4318 )

same across languages
```

Sampling

- Sampling decides how many traces you actually keep, trading total visibility against cost. The right strategy keeps the traces that matter, like errors and slow requests, without storing every healthy one.
 - Always on sampling
 - Probabilistic sampling
 - Tail-based sampling
 - Parent-based sampling

■ Sampling strategies

Always-on

Probabilistic

Tail-based

Parent-based

Head vs Tail Sampling

- Head sampling decides at the start of a request, before you know how it ends; tail sampling decides after the trace completes. Tail sampling is pricier but can deliberately keep every error and slow trace.
 - Head sampling is fast
 - Tail sampling sees full trace
 - Tail sampling is more expensive
 - Tail sampling enables filtering

■ Head vs tail sampling

Head sampling

- decides at the start
- fast and cheap
- cannot see the end

Tail sampling

- decides after the trace
- keeps errors and slow
- more expensive

Resource Attributes

- Resource attributes describe who is emitting the telemetry: service name, version, environment, region. They are what let you filter a sea of traces down to one service in production.
 - Service name
 - Environment
 - Region
 - Version

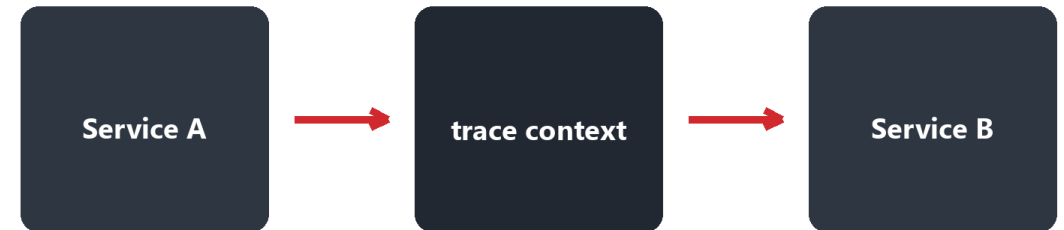
Resource attributes

```
service.name = checkout
deployment.environment = prod
service.version = 1.4.2
cloud.region = us-east-1
```

Context Propagation

- Context propagation is how a single trace survives a hop between services. The trace identifiers travel with the request, so a downstream service joins the same trace instead of starting a new one.
 - Trace context travels with requests
 - Implemented via headers
 - Works across protocols
 - Required for distributed tracing

■ Context propagation



the same trace spans both

Trace Context Headers

- That context usually rides in standardized HTTP headers. The W3C traceparent and tracestate headers are understood across vendors, which is what makes cross-service tracing just work.
 - traceparent header
 - tracestate header
 - W3C standard
 - Supported across vendors

Trace context headers

```
traceparent: 00-<trace-id>-<span-id>-01
tracestate: vendor=value
```

W3C standard headers

What Is Baggage

- Baggage is small key-value metadata that rides along with a trace, like a customer tier or request ID. It carries business context across services, but keep it small since it travels on every hop.
 - Propagates across services
 - Used for custom metadata
 - Not used for metrics
 - Should be kept small

What is baggage

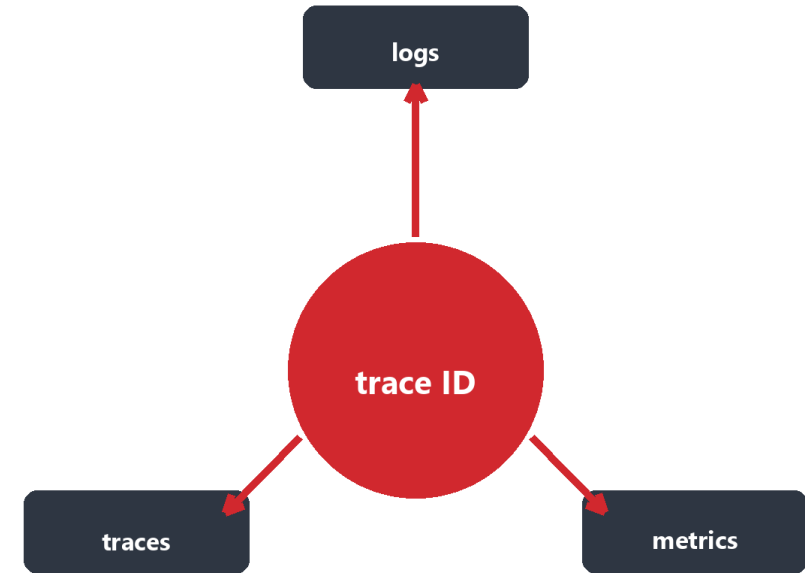
```
baggage:  user.tier=gold
          request.id=abc123

keep it small, travels everywhere
```

Correlation

- Correlation is what lets you jump between signals without losing your place. With the trace ID stamped into logs and metrics, you can move from a graph to a trace to the exact log line.
 - Trace ID in logs
 - Span ID in logs
 - Metrics tagged with trace data
 - Enables cross-signal navigation

■ Correlation

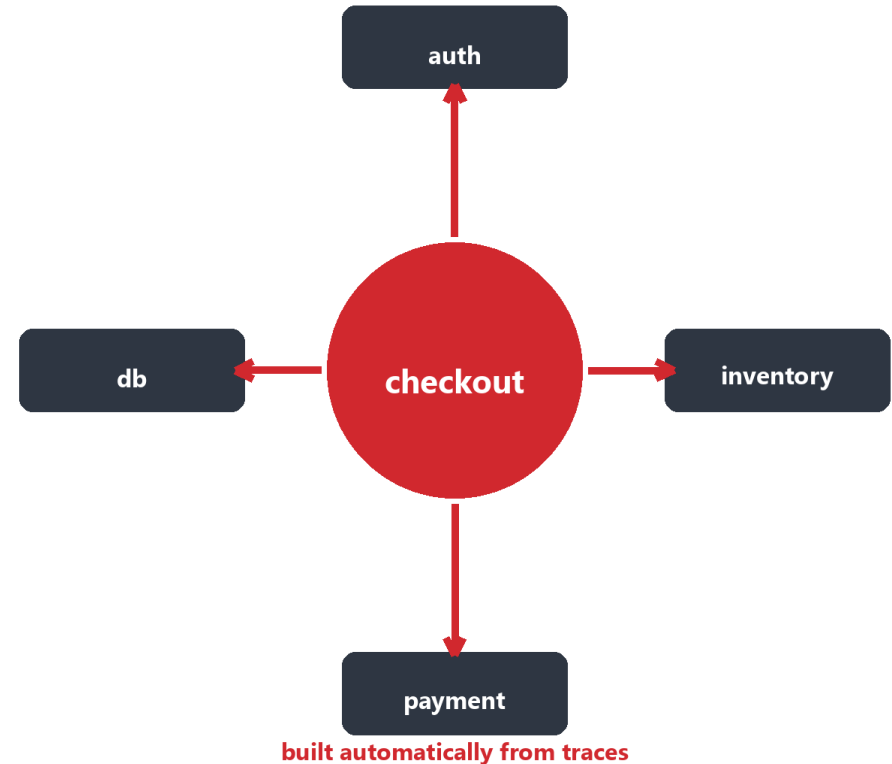


jump between signals

Service Graphs

- A service graph is a live map of which services call which, built automatically from traces. It shows dependencies, traffic, and error rates, so a bottleneck is visible at a glance.
 - Shows dependencies
 - Shows request volume
 - Shows error rates
 - Shows latency edges

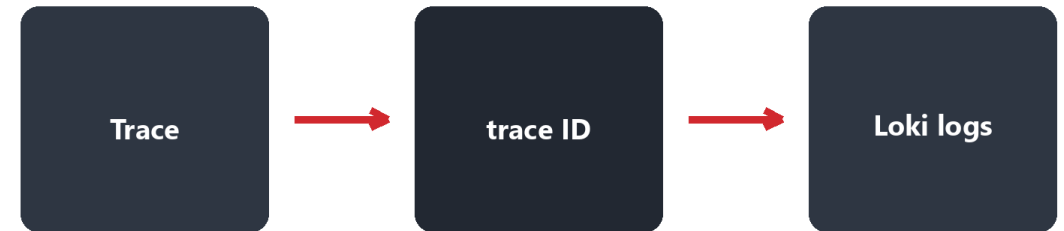
■ Service graphs



Trace to Logs

- Trace to logs lets you jump from a span straight into the relevant log lines. Using the trace ID as a filter into Loki turns which logs go with this request from minutes into one click.
 - Uses trace ID
 - Works with Loki
 - Filters logs automatically
 - Improves debugging speed

■ Trace to logs

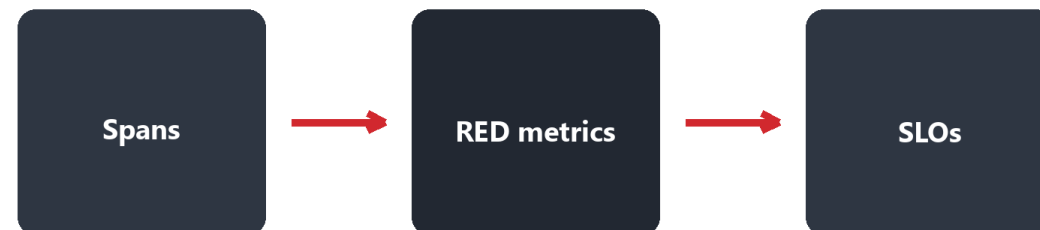


from request to raw events

Trace to Metrics

- Trace to metrics connects one request to the aggregate picture. Span data can generate RED metrics, so the same instrumentation powers both deep dives and your SLO dashboards.
 - Uses span attributes
 - Generates RED metrics
 - Supports SLOs
 - Supports capacity analysis

■ Trace to metrics



RED Method

- The RED method watches a service through three numbers: Rate, Errors, and Duration. Derived straight from trace data, it is a simple, consistent way to judge user-facing health.
 - Rate
 - Errors
 - Duration
 - Derived from traces

The RED method



derived from traces

USE Method

- The USE method looks at resources instead of requests: Utilization, Saturation, and Errors. It complements RED by telling you whether the infrastructure underneath is healthy.
 - Utilization
 - Saturation (CPU queue, swap space)
 - Errors
 - Infrastructure focused

The USE method

Utilization

Saturation

Errors

infrastructure focused

OTEL in Incident Response

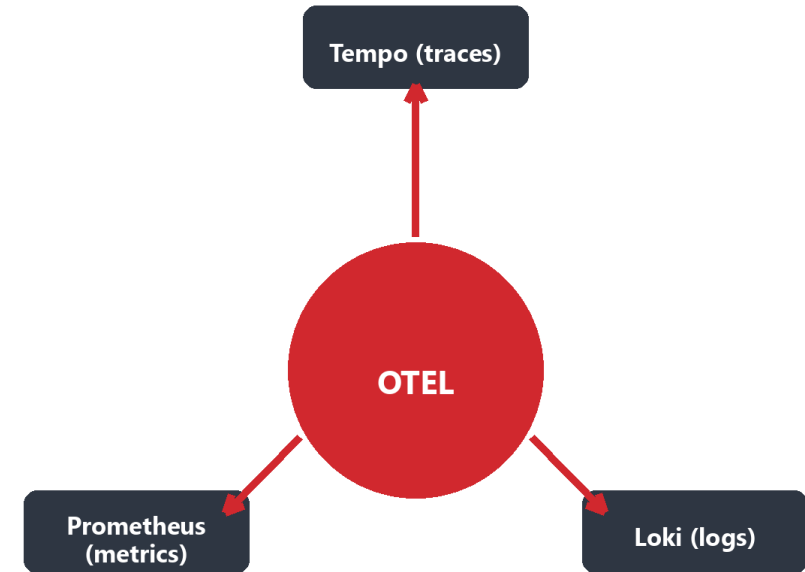
- In an incident, OTEL turns guesswork into evidence. You find the failing service, follow the request path, inspect the errors, and confirm the fix with live traces.
 - Find failing service
 - Follow request path
 - Inspect errors
 - Confirm fix with live traces



OTEL and the LGTM Stack

- OpenTelemetry is the data plane that feeds the LGTM stack. It sends traces to Tempo, logs to Loki, and metrics to Prometheus or Mimir, all from one instrumentation layer.
 - OTEL generates telemetry
 - Tempo stores traces
 - Loki stores logs
 - Prometheus or Mimir stores metrics

■ OTEL and the LGTM stack



Tempo in the LGTM Stack

- Tempo is the tracing backend, built to store huge volumes of traces cheaply. It receives OTLP and integrates with Grafana, so traces sit right beside your logs and metrics.
 - Receives OTLP
 - Stores trace data
 - No indexing by default
 - Optimized for cost

Tempo in the LGTM stack

✓ Receives OTLP

✓ Stores traces (Parquet)

✓ No full-text index

✓ Searched with TraceQL

Why Tempo Does Not Index

- Tempo deliberately skips a traditional full-text index to keep storage cheap and ingestion fast. Instead it stores traces in a columnar format that TraceQL scans directly, so you can still search by service, duration, and attributes.
 - Lower storage cost
 - Faster ingestion
 - Relies on metadata
 - Optimized for large volumes

Why Tempo does not index

Traditional index

- index every span
- powerful but costly

Tempo

- columnar + TraceQL
- cheap and fast

Grafana as the Control Plane

- Grafana is the one interface that ties traces, logs, and metrics together. From a single screen you correlate signals and navigate from a graph into a trace without switching tools.
 - Unified dashboards
 - Explore view
 - Trace navigation
 - Signal correlation



Trace Exploration in Grafana

- Grafana lets you explore traces interactively, not just view them. You search by service or duration, open the span tree, and drill into the span that errored or ran long.
 - Search by service
 - Search by duration
 - View span trees
 - Inspect errors

Trace exploration in Grafana

GET /api

service A

service B

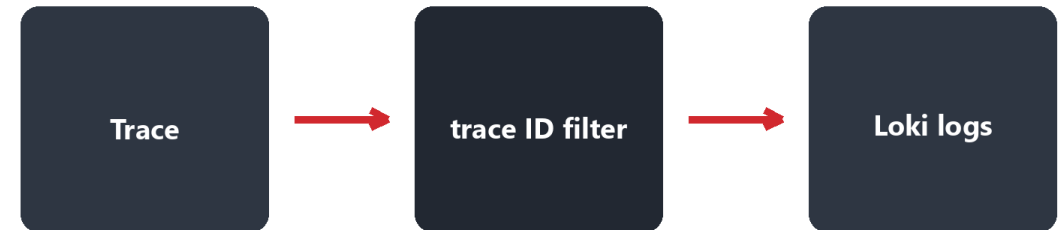
db

drill into the slow span

Trace to Logs in LGTM

- Inside the LGTM stack, a trace links straight to its logs in Loki. The trace ID becomes a log filter automatically, so you go from a slow request to its raw events in seconds.
 - Trace ID used as filter
 - Automatic log queries
 - Reduces investigation time
 - Improves debugging flow

■ Trace to logs in LGTM



Trace to Metrics in LGTM

- LGTM also turns trace attributes into metrics. RED metrics, latency percentiles, and error rates come from spans, which is how individual requests roll up into SLOs.
 - RED metrics from spans
 - Latency percentiles
 - Error rates
 - Throughput

Trace to metrics in LGTM

RED metrics

Latency p95

Error rate

Throughput

Service Graphs in LGTM

- Service graphs in Grafana are generated from trace data to map your system. They show call relationships and traffic, and highlight the edges where latency or errors concentrate.
 - Auto-generated
 - Shows call relationships
 - Shows traffic volume
 - Highlights bottlenecks

■ Service graphs in LGTM

✓ Auto-generated

✓ Call relationships

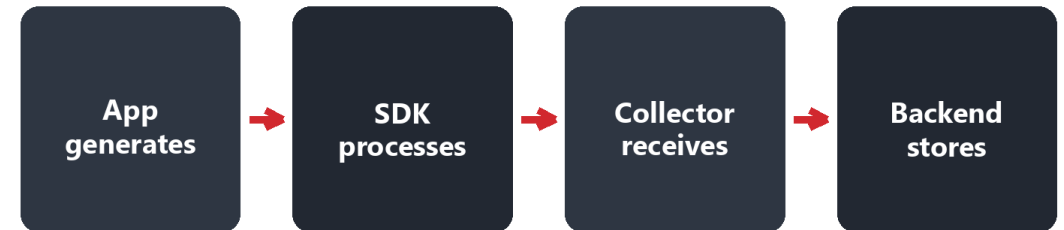
✓ Traffic volume

✓ Highlights bottlenecks

OTEL Data Flow

- It is worth holding the whole path in your head: the app generates telemetry, the SDK processes it, the collector receives it, and a backend stores it. That pipeline itself needs to be reliable.
 - App generates telemetry
 - SDK processes data
 - Collector receives data
 - Backends store data

■ OTEL data flow



Why Platform Teams Care

- Platform teams adopt OTEL to offer observability as a shared service. Standard telemetry means new teams are observable on day one, with consistent tooling and less onboarding.
 - Standardized telemetry
 - Reduced onboarding time
 - Consistent tooling
 - Improved reliability



OTEL in FastAPI

- FastAPI can be auto-instrumented with OpenTelemetry to trace every request, with no changes to your handlers. The ASGI middleware creates a span per endpoint and captures request metadata for you.
 - Auto-instrumentation supported
 - Works with ASGI middleware
 - Creates spans per endpoint
 - Captures request metadata

OTEL in FastAPI

```
opentelemetry-instrument \  
  uvicorn app:api
```

a span per endpoint, no code changes

Auto-Instrumentation

- Auto-instrumentation injects telemetry into common libraries at runtime, so you do not hand-write spans for routine work. Configured at startup, it covers frameworks, HTTP clients, and databases out of the box.
 - Uses monkey patching
 - Configured at startup
 - Minimal code changes
 - Covers common frameworks

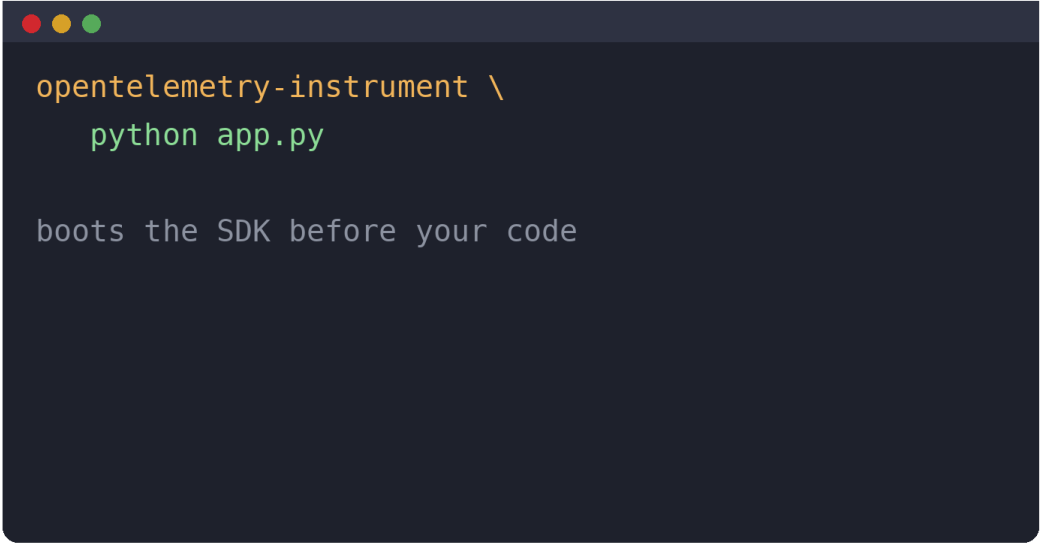
Auto-instrumentation

- ✓ Injects at runtime
- ✓ Configured at startup
- ✓ Minimal code changes
- ✓ Covers common frameworks

opentelemetry-instrument

- The opentelemetry-instrument command wraps your app's startup and boots the SDK before your code runs. It loads the right instrumentation packages, so tracing is on without editing main().
 - CLI wrapper
 - Initializes providers
 - Loads instrumentation packages
 - Requires correct dependencies

opentelemetry-instrument



```
opentelemetry-instrument \  
python app.py  
  
boots the SDK before your code
```

OTLP Endpoints

- An OTLP endpoint is simply where telemetry is sent, over gRPC on 4317 or HTTP on 4318. It usually points at a collector, and if the app cannot reach it you get no traces, so connectivity matters.
 - HTTP or gRPC
 - Usually port 4317 or 4318
 - Collector or backend
 - Critical for connectivity

OTLP endpoints

```
OTEL_EXPORTER_OTLP_ENDPOINT
gRPC  -> :4317
HTTP  -> :4318
```

Custom Spans

- Auto-instrumentation covers the plumbing; custom spans cover your business logic. Wrapping a critical code path in a span, with your own attributes, is how you expose a bottleneck the framework cannot see.
 - Manually defined
 - Used for critical code paths
 - Add custom attributes
 - Expose performance bottlenecks

Custom spans

```
with tracer.start_as_current_span(  
    "process_order") as span:  
    span.set_attribute("order.id", id)
```

Error Tracing

- When an uncaught exception propagates out of an instrumented span, OTEL records it and sets the span status to error. That status makes failures searchable in Tempo, so you can pull up every trace that hit the bug.
 - Exception recorded in span
 - Status set to error
 - Searchable in Tempo
 - Supports incident analysis

Error tracing

```
except Exception as e:
    span.record_exception(e)
    span.set_status(Status(ERROR))
```

Outbound Calls

- Tracing gets powerful when it follows calls out of your service. With HTTP, database, and message-queue instrumentation, the trace shows dependency latency, not just your own code.
 - HTTP client instrumentation
 - Database instrumentation
 - Message queue instrumentation
 - Shows dependency latency

■ Outbound calls



Sampling in Production

- Almost no one keeps 100 percent of traces in production; the volume and cost are too high. The usual move is to sample lightly but keep the interesting traces, the errors and the slow ones.
 - Lower cost
 - Reduced storage
 - Focus on errors
 - Tail-based filtering

■ Sampling in production

✓ Rarely sample 100%

✓ Lower cost and storage

✓ Keep the errors

✓ Tail-based filtering

Performance Overhead

- Well-configured OTEL adds little overhead, but it is not free. CPU, network, and storage cost grows with how much you sample and export, which is why sampling strategy matters.
 - CPU overhead
 - Network overhead
 - Storage overhead
 - Generally acceptable in prod

■ Performance overhead

✓ CPU overhead

✓ Network overhead

✓ Storage overhead

✓ Generally acceptable

Common Production Patterns

- In production, OTEL usually runs with a collector and central config rather than ad hoc per app. Sidecar or DaemonSet collectors plus central sampling rules keep it manageable as you scale.
 - Sidecar collectors
 - DaemonSet collectors
 - Central sampling rules
 - Multi-backend routing

Common production patterns

Sidecar collector

DaemonSet collector

Central sampling

Multi-backend routing

MCQ 1

- What is the main purpose of OpenTelemetry?
- A. Store logs and metrics
- B. Provide dashboards and alerts
- C. Generate and export telemetry
- D. Replace Prometheus



MCQ 1 – Answer

- What is the main purpose of OpenTelemetry?
- A. Store logs and metrics
- B. Provide dashboards and alerts
- **C. Generate and export telemetry**
- D. Replace Prometheus



MCQ 2

- Which component is responsible for receiving and forwarding telemetry out of process?
- A. SDK
- B. Instrumentation library
- C. Collector
- D. Exporter



MCQ 2 – Answer

- Which component is responsible for receiving and forwarding telemetry out of process?
- A. SDK
- B. Instrumentation library
- **C. Collector**
- D. Exporter



MCQ 3

- What does context propagation enable?
- A. Log aggregation
- B. Metric sampling
- C. Distributed tracing across services
- D. Alert routing



MCQ 3 – Answer

- What does context propagation enable?
- A. Log aggregation
- B. Metric sampling
- **C. Distributed tracing across services**
- D. Alert routing



MCQ 4

- Why is tail-based sampling powerful?
- A. It samples faster
- B. It reduces CPU usage
- C. It sees the full trace before deciding
- D. It eliminates exporters



MCQ 4 – Answer

- Why is tail-based sampling powerful?
- A. It samples faster
- B. It reduces CPU usage
- **C. It sees the full trace before deciding**
- D. It eliminates exporters



MCQ 5

- Which LGTM component stores distributed traces?
- A. Loki
- B. Prometheus
- C. Grafana
- D. Tempo



MCQ 5 – Answer

- Which LGTM component stores distributed traces?
- A. Loki
- B. Prometheus
- C. Grafana
- **D. Tempo**

